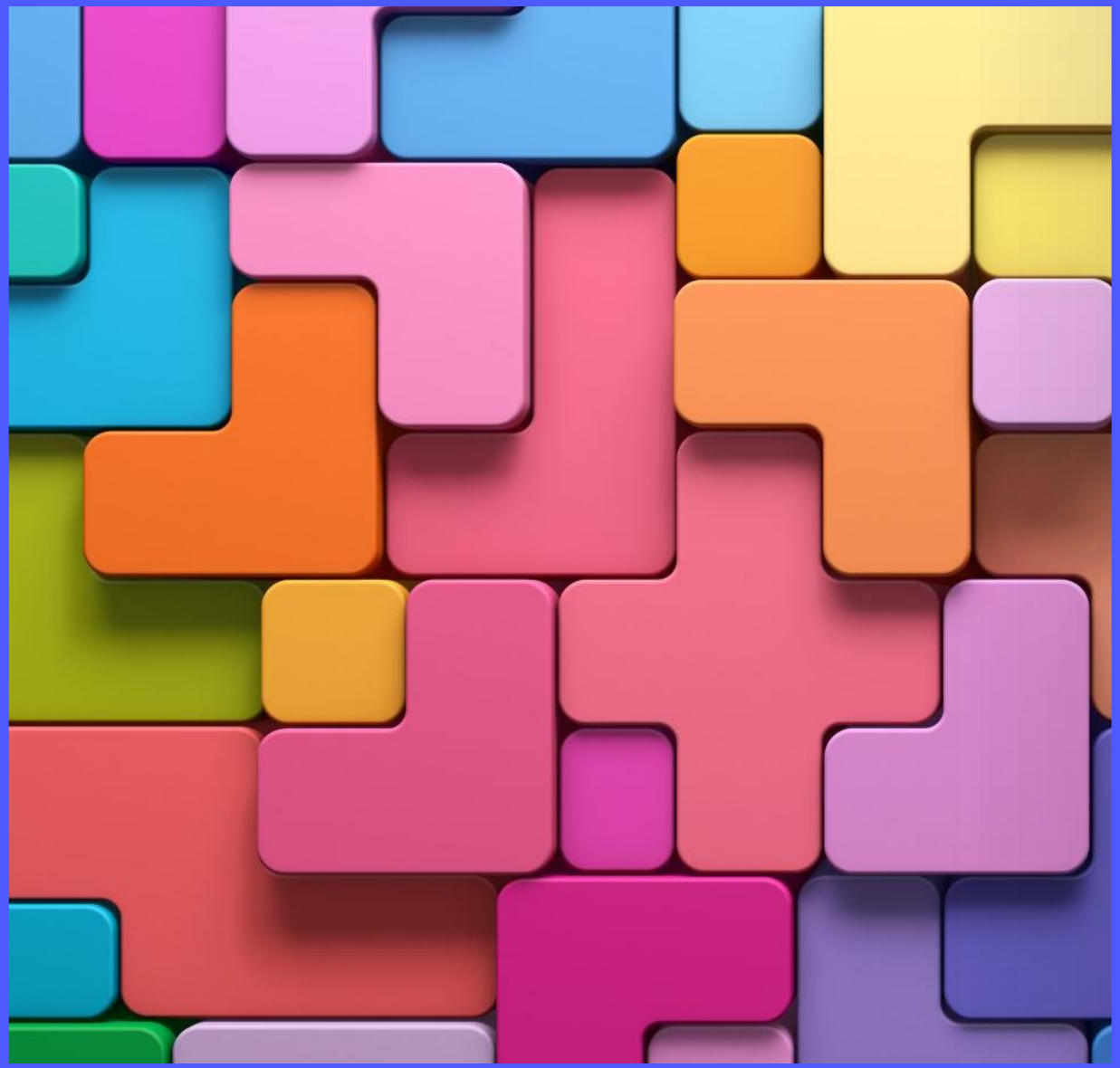


คณิตศาสตร์ในการโปรแกรม

Math in Programing

รวบรวมโดย อ.ดร.ฐาปนพงษ์ รักกาญจนันท์

คณิตศาสตร์ช่วยอะไรเราได้บ้าง

- ช่วยหาสมการสำเร็จรูป
- ช่วยหา Pattern เพื่อลดรูปสมการ
- ในการแข่ง ICPC (โดยเฉพาะในเอเชีย) จะมีโจทย์คณิตศาสตร์ 1 – 2 ข้อ
- ในการแข่ง IOI ไม่ค่อยมีคณิตศาสตร์บริสุทธิ์แต่จะมีบางท่าที่ต้องใช้แนวคิดคณิตศาสตร์
  - ค่ายสองก็เช่นกัน ^___^

วอร์มอัพกันหน่อย

- จำเรื่องการเช็คว่าตัวเลขที่ให้เป็นจำนวนเฉพาะได้ไหม
- ถ้าเราใช้ Loop เราต้อง Loop ตั้งแต่เท่าไหร่ถึงเท่าไหร่

ทำไมถึงแค่ sqrt(n)?

- ถ้า n ไม่ใช่จำนวนเฉพาะแสดงว่า n = a*b
- ถ้า a และ b มากกว่า sqrt(n) นั่นแหละว่า a*b มากกว่า n ซึ่งขัดแย้งกับสมการข้างต้น
- นั่นแปลว่า a หรือ b ต้องน้อยกว่า sqrt(n) ดังนั้นเราจึง Loop ถึงแค่ sqrt(n)

อัพเกรดขึ้นมานิด

- ถ้าสมมติเราต้องการเขียนเลขจำนวนเฉพาะที่อยู่ระหว่าง 2 ถึง n
- ง่ายสุดคือ Loop 2 ถึง n แล้วเช็คว่าตัวไหนเป็นจำนวนเฉพาะ
  - ใช้ Big-Oh เท่าไหร่นะ

Sieve of Eratosthenes

- เราทำให้เร็วขึ้นได้ด้วยอัลกอริทึมที่ชื่อ Sieve of Eratosthenes (คำหลังอ่านว่าเอราโตสเทแนส)
  - เก็บ Array ของ Boolean ขนาด n+1 (Array[x] = True ถ้า x เป็นจำนวนเฉพาะ)
  - เริ่มต้นด้วยการตั้งค่า Array[0] = Array[1] = False และ Array[2…] = True
  - เลือกจำนวนเฉพาะตัวถัดไป (ตัวแรกคือเลข 2) จากนั้นปรับ Array[2*i] ทั้งหมดให้เป็น False
  - ทำต่อจนไม่เหลือจำนวนเฉพาะตัวถัดไป

ลองหน่อยสิ (0 – 99)

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49
50	51	52	53	54	55	56	57	58	59
60	61	62	63	64	65	66	67	68	69
70	71	72	73	74	75	76	77	78	79
80	81	82	83	84	85	86	87	88	89
90	91	92	93	94	95	96	97	98	99

ลองเขียนโค้ดหน่อย

- รับค่า n
- พิมพ์จำนวนเฉพาะที่มีค่าตั้งแต่ 2 ถึง n โดยใช้วิธี Sieve of Eratosthenes
- เราจะไปต่อเมื่ออย่างน้อย 20 คนเขียนเสร็จ

ใครอยากลองวิชา

- C2Q002 ข้อสันนิษฐานของโกลด์บาค
- ให้จำนวนคู่ n ที่มีค่ามากกว่าหรือเท่ากับ 4 จงหาจำนวนเฉพาะ (ในสมัยนั้นนับเลข 1 เป็นจำนวนเฉพาะด้วย) สองจำนวนที่บวกกันแล้วได้ n ถ้าหาไม่ได้ให้พิมพ์ Goldbach’s conjecture is wrong ถ้ามีหลายคำตอบให้พิมพ์คู่ที่ผลต่างสูงสุด
- ตัวอย่าง รับค่า 8 ให้พิมพ์ 8 = 3 + 5

หารร่วมมาก คูณร่วมน้อย (ทบทวน)

- ตัวหารร่วมมากของ a และ b คือจำนวนที่มากที่สุดที่หาร a และ b ลงตัว
- วิธีการหาตัวหารร่วมมาก เราเคยเรียนไปแล้ว คืออัลกอริทึมของยุคลิด
- เราสามารถเขียนอัลกอนี้ในบรรทัดเดียวได้ (จำไว้เลย ได้ใช้แน่ในอนาคต)

```
int gcd(int a, int b) { return (b == 0 ? a : gcd(b, a % b)); }
int lcm(int a, int b) { return (a * (b / gcd(a, b))); }
```

- คูณร่วมน้อยคือจำนวนที่น้อยที่สุดที่ a และ b หารลงตัว

เศษส่วน

- ปกติแล้วในการเขียนโปรแกรมคอมพิวเตอร์ โจทย์ส่วนมากจะใช้เลขจำนวนเต็มเป็นหลัก อย่างน้อยข้อมูลเข้าก็เป็นจำนวนเต็ม
- แต่บางโอกาสก็จะเอานำจำนวนเต็มในรูปของเศษส่วนมาใช้งาน
- เศษส่วน $\frac{a}{b}$ เรียกว่าเศษส่วนแท้ ถ้า a และ b ไม่มีตัวประกอบร่วม
- ถามว่าเราจะแปลงเศษส่วนที่ไม่แท้เป็นเศษส่วนแท้อย่างไร?

อนุกรม Farey

- อนุกรม Farey อันดับ n คืออนุกรมของเศษส่วนแท้ (มากกว่า 0 แต่ไม่ถึง 1) ที่มีตัวส่วนไม่เกิน n เรียงลำดับจากน้อยไปมาก
  - อนุกรม Farey อันดับ 5 คือ $\frac{1}{5}, \frac{1}{4}, \frac{1}{3}, \frac{2}{5}, \frac{1}{2}, \frac{3}{5}, \frac{2}{3}, \frac{3}{4}, \frac{4}{5}, \frac{1}{1}$
- คำถามคือถ้าให้ค่า n มา คุณจะหาพจน์ที่ k ของอนุกรม Farey อันดับ n ได้อย่างไร

Construction Problem

- เราพบว่าโจทย์ข้อนี้บังคับให้เราสร้างข้อมูลขึ้นมาเพื่อใช้ประโยชน์แค่ข้อมูลเดียว
  - Sieve of Eratosthenes ก็เป็นโจทย์ในลักษณะเดียวกัน
- เพราะฉะนั้นเราจะต้องเขียนโปรแกรมที่สร้างเศษส่วนแท้ขึ้นมาก่อน จากนั้นค่อยนำมาเรียงกัน
- เศษส่วนแท้ตามเงื่อนไขสร้างอย่างไรดี

Brute Force Algorithm

- Brute Force Algorithm คืออัลกอริทึมที่ค้นหาทุกวิธีที่เป็นไปได้แบบง่าย
- ในโจทย์นี้เราสามารถ Loop ทั้งเศษและส่วน แล้วค่อยกรองเอาเศษส่วนไม่แท้ด้วยการหาตัวหารร่วมมาก

```
for (int denominator = 2; denominator <= n; denominator++)
    for (int numerator = 1; numerator < denominator; numerator++)
        if (gcd(denominator, numerator) == 1) // this is proper fraction
```

Smart Construction

- แทนที่จะสร้างมาทุกตัวแล้วเรียงกัน เราสามารถหาสมการที่หาพจน์ถัดไปได้เลยไหม
- ลองสังเกตดีๆ
  - $\frac{1}{5}, \frac{1}{4}, \frac{1}{3}, \frac{2}{5}, \ldots$
  - $\frac{1}{4} = \qquad , \frac{1}{3} =$

Farey Addition

- ลองเขียนสมการดู
- สมมติว่าเรามีพจน์ $\frac{a}{b}, \frac{c}{d}$ แล้วเราต้องการหาพจน์ถัดไป $\frac{p}{q}$
- ลองเขียนสมการของ p และ q ดู

ใครอยากส่งคำตอบ

- มีโจทย์ C2Q003 ลำดับแฟเรย์
- ให้ค่า n และ k มาให้ (มีหลายเทสเคส)
  - ตอบพจน์ที่ k ของอนุกรม Farey ลำดับที่ n
- ตัวอย่าง รับค่า 5 5 ให้พิมพ์ 1/2
- ตัวอย่าง รับค่า 5 1 ให้พิมพ์ 1/5

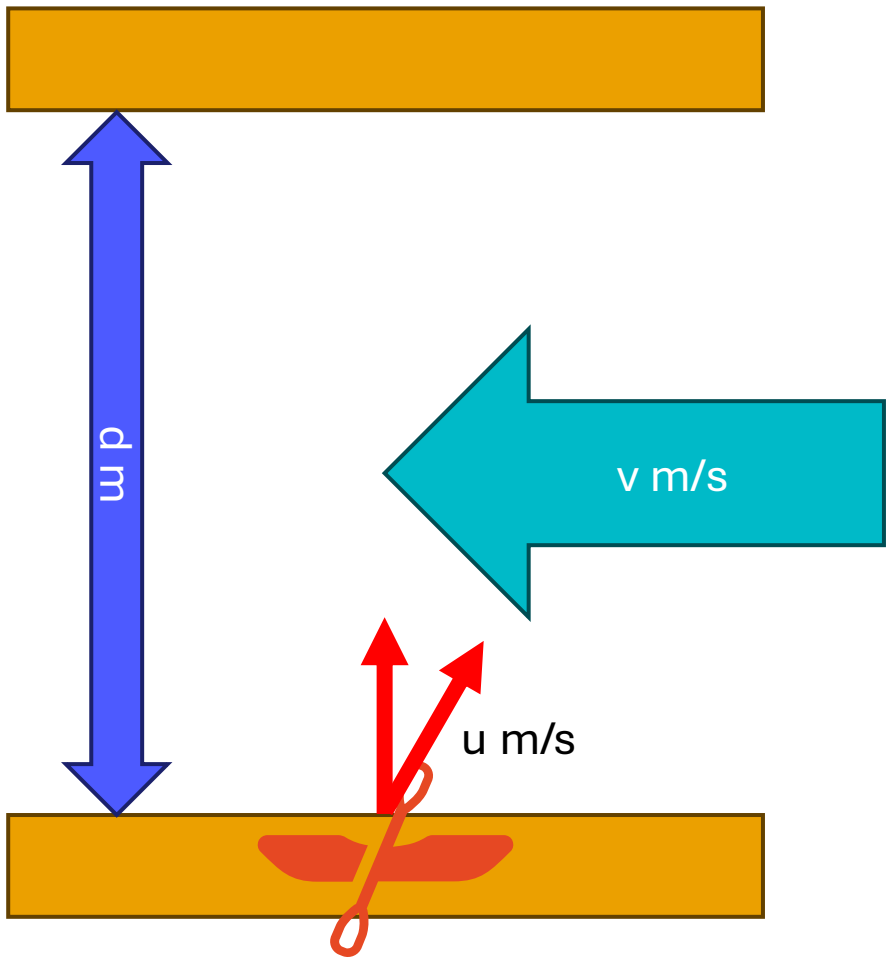
```
while (cin >> n >> k) {
    ...
}
```

คณิตศาสตร์เป็นภาษาของวิทยาศาสตร์

- ในบางครั้งเราอาจจะเจอปัญหาแนววิทยาศาสตร์ แต่ข้างในแล้วมันก็จะเป็นวิทยาศาสตร์คำนวณ
- การเข้าใจคณิตศาสตร์ในวิทยาศาสตร์ช่วยให้เราเข้าใจปัญหาได้ง่ายขึ้น

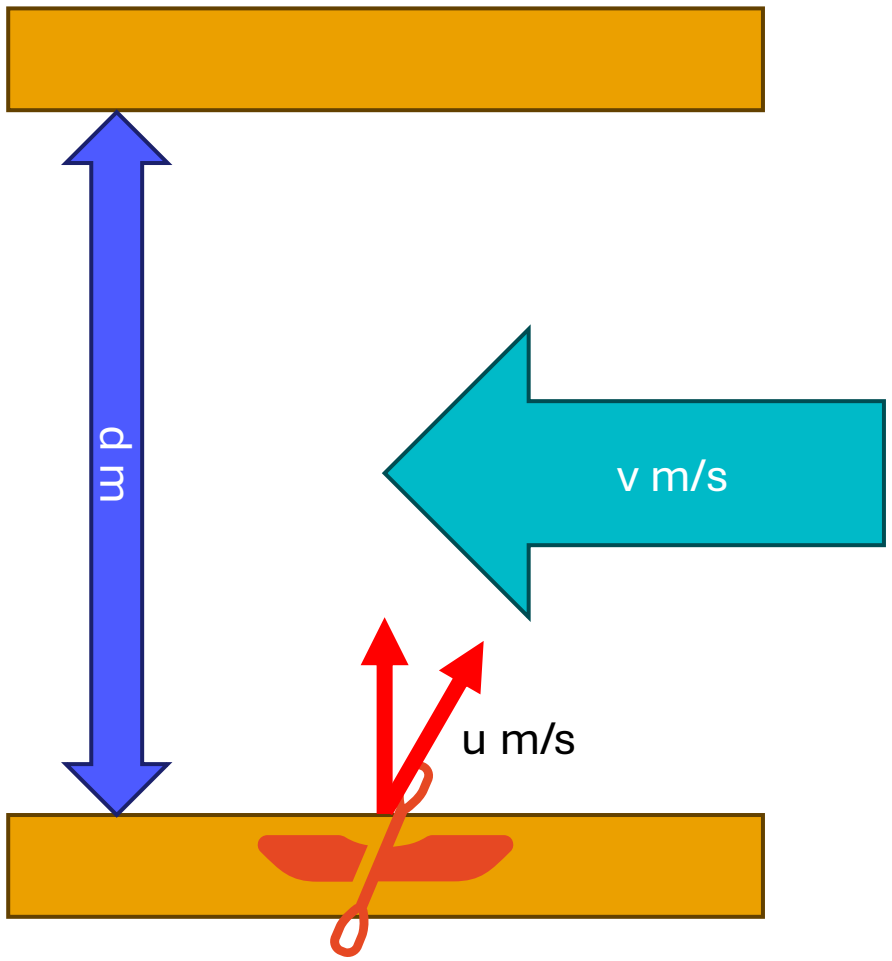
พายเรือข้ามคลอง

- สมมุติว่าเราต้องการพายเรือข้ามคลองที่มีความกว้าง d เมตร
- เราสามารถพายเรือในทิศทางไหนก็ได้ที่ความเร็ว u เมตรต่อวินาที
- กระแสน้ำในคลองมีความเร็ว v เมตรต่อวินาที



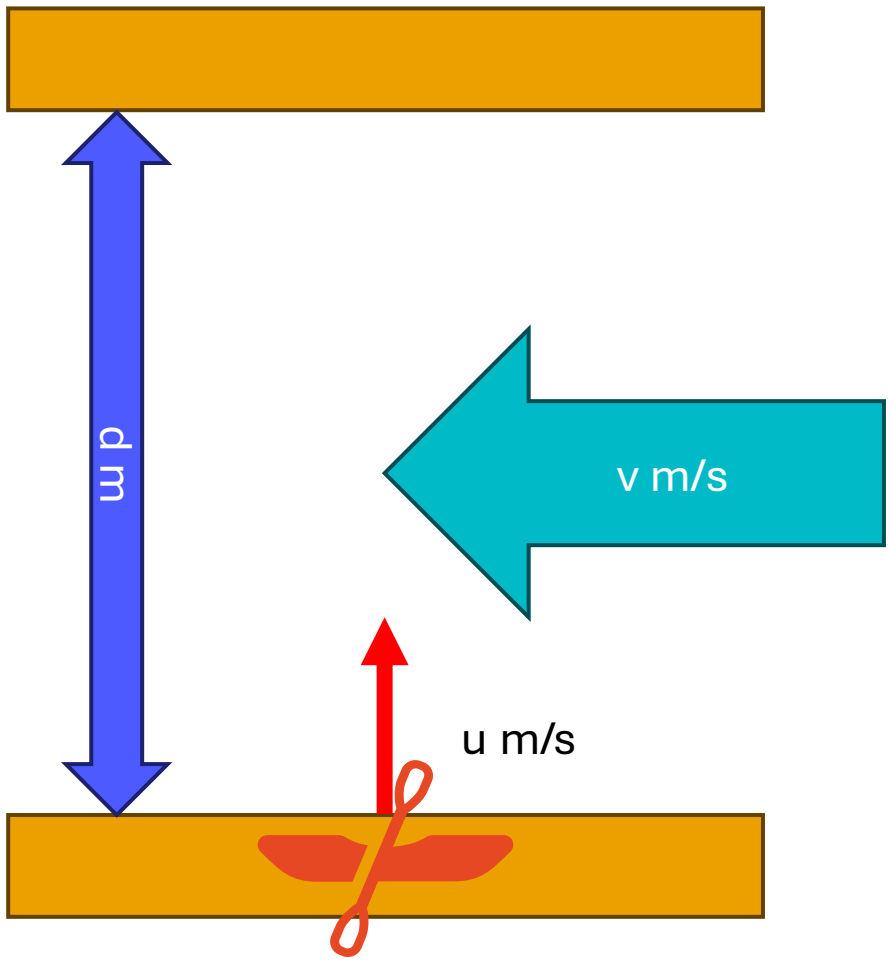
พายเรือข้ามคลอง

- เราจะเปรียบเทียบวิธีการพายสองแบบ
  - พายแบบตรง ๆ ไม่สนกระแสน้ำ
  - พายให้ทิศทางเรือตั้งฉากกับกระแสน้ำตลอดเวลา
- คำถามคือเราสามารถหาวิธีพายทั้งสองแบบนี้ที่ต้องแตกต่างกันได้ไหม



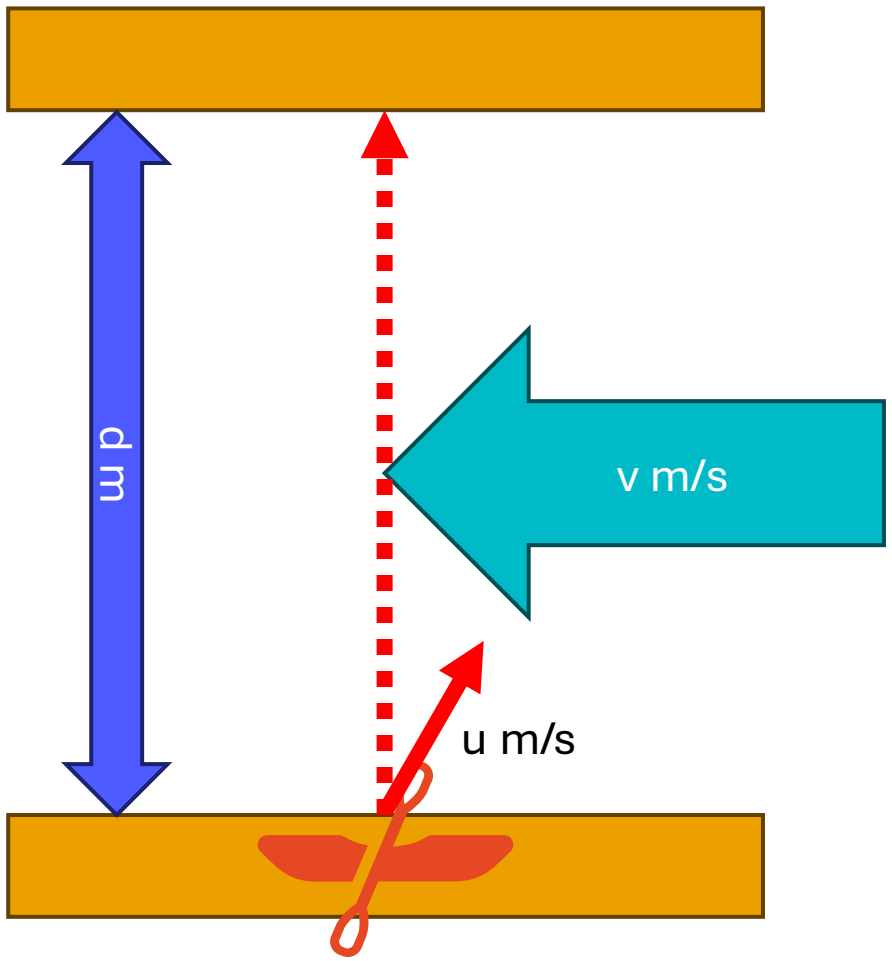
คำถามง่ายก่อน

- ถ้าเราพายแบบตรง ๆ ไม่สนกระแสน้ำ ใช้เวลาเท่าไหร่



คำถามยาก

- พายยังไงให้ทิศทางเรือตั้งฉากกับกระแสน้ำเสมอ
  - ความเร็วคงที่ เหลือแค่องศา
  - จำเป็นต้องหาองศาไหม?
- ถ้าทำได้ ใช้เวลาเท่าไหร่



ใครอยากส่งคำตอบ

- มีโจทย์ C2Q004 พายเรือข้ามคลอง
- ให้ค่า d, u, v มาให้ (มีหลายเทสเคส)
  - ถ้าหาวิธีพายสองวิธีที่ต่างกันได้ให้แสดงผลต่างของเวลา (ทศนิยมสามตำแหน่ง)
  - ถ้าหาไม่ได้ให้พิมพ์ can’t determine
- ตัวอย่าง รับค่า 8 5 6 ให้พิมพ์ Case 1: 1.079

```
int test = 1;
double dt = 1.079123;
printf("Case %d: %.3lf\n", test, dt);
```

ยกกำลัง

- การยกกำลังก็เป็นอีกหนึ่ง Operation ที่พบได้ในการแข่งเขียนโปรแกรม
- โดยปกติแล้วถ้าเราจะเขียนฟังก์ชันเลขยกกำลัง เราอาจจะเขียนแบบนี้ได้

```
int normapExp(int base, int p) {
    int result = 1;
    for (int i = 0; i < p; i++) result *= base;
    return result;
}
```

Divide and Conquer

- แต่เราสามารถทำให้การยกกำลังไวขึ้นได้ด้วยการแบ่งเลขออกเป็นผลคูณของสองตัวที่มีเลขยกกำลังเท่ากัน
- แบบปกติ: $2^9 = 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \approx O(p)$
- แบบไว: $2^9 = 2^8 \times 2 = (2^4)^2 \times 2 = \left((2^2)^2\right)^2 \times 2 \approx O(\log p)$

Efficient Exponentiation

- หลักการสำคัญมีดังนี้
  - เคสง่าย $A^0 = 1$
  - ถ้า p เป็นเลขคู่ เราจะได้ว่า $A^p = \left(A^{\frac{p}{2}}\right)^2$
  - ถ้า p เป็นเลขคี่ เราจะได้ว่า $A^p = A^{p-1} \times A$

```
int fastExp(int base, int p) {
    if (p == 0) return 1;
    if (p % 2 == 0) {
        int half = fastExp(base, p / 2);
        return half * half;
    } else {
        return base * fastExp(base, p - 1);
    }
}
```

Square Matrix Exponentiation

- เราสามารถใช้เทคนิคเดียวกันในการหาค่ายกกำลังของ Matrix
- $A = \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix}$
- $A^5 = (A^2)^2 \times A = \left(\begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix}^2\right)^2 \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix} = \begin{bmatrix} -1 & -1 \\ 2 & -2 \end{bmatrix}^2 \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix}$
- $= \begin{bmatrix} -1 & 3 \\ -6 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix} = \begin{bmatrix} 5 & 1 \\ -2 & 6 \end{bmatrix}$

Fibonacci Sequence

- ถ้าเราจำได้ อนุกรม Fibonacci มีสมการแบบนี้
  - $F(n) = F(n-1) + F(n-2)$
- ถ้าจะหาพจน์ที่ n ของอนุกรม Fibonacci เราเริ่มต้นจากหาพจน์ที่ 1 และค่อย ๆ เลื่อนไปจนถึงพจน์ที่ n

Fibonacci Matrix

- เราสามารถเขียนสมการ Fibonacci ให้อยู่ในรูป Matrix ได้ดังนี้
  - $\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} & \\ & \end{bmatrix} \begin{bmatrix} F_{n-1} \\ F_{n-2} \end{bmatrix}$
- เพราะฉะนั้น เราจะได้สมการรวมเป็น
  - $\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} & \\ & \end{bmatrix}^{n-1} \begin{bmatrix} F_1 \\ F_0 \end{bmatrix}$

ใครอยากส่งคำตอบ

- มีโจทย์ C2Q005 ฟีโบนัชชีแบบมอดูลาร์
- ให้ค่า n, m มาให้ (มีหลายเทสเคส)
  - ตอบเป็น $F_n \ mod \ 2^m$
- ไม่ยากมาก ทริคคือ mod ทุกการคูณและบวก และใช้ long long แทน int

```
long long temp[2][2] = {0};
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 2; j++) {
        long long sum = 0;
        for (int k = 0; k < 2; k++)
            sum += (A[i][k] * A[k][j]) % MOD;
        temp[i][j] = sum % MOD;
    }
}
```